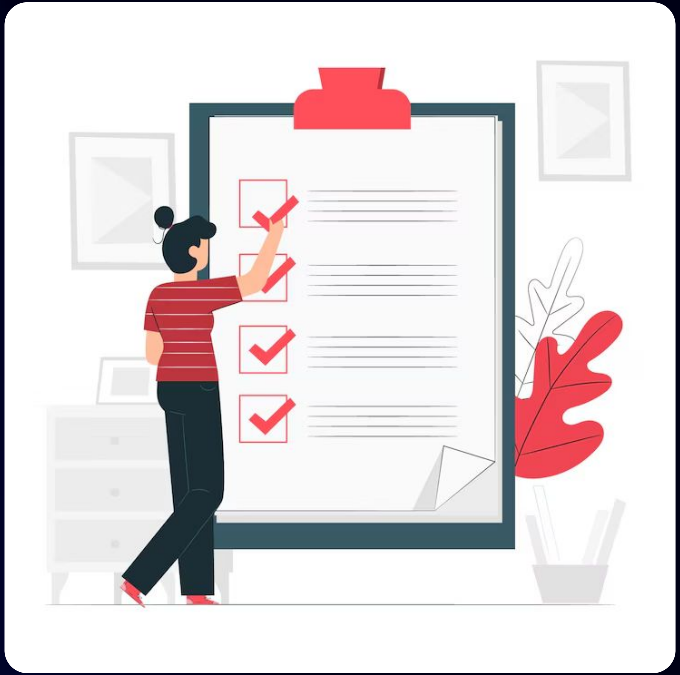
GitHub & Remote Collaboration Workflow

Learning Objectives

- Understand Git vs GitHub
- Set up a GitHub account and profile
- Create and connect remote repositories
- Push and clone code
- Keep code updated using pull and fetch
- Learn basic collaboration using branches
- Explore GitHub Desktop (optional)

Prerequisites

- Basic understanding of programming concepts
- Familiarity with file and folder structure (directories)
- Basic knowledge of using a computer terminal/command line
- Understanding of how files are created, edited, and saved
- Basic knowledge of Git fundamentals (init, add, commit)
- Awareness of Git architecture (working directory, staging, repository)
- Willingness to learn and experiment with new tools



Git vs GitHub
(Clear Distinction)

Introduction to Git vs GitHub

Git

Local version control
Tracks changes on your machine.

Primarily individual
Designed for single developer use.

Offline capable
Works without an internet connection.

Code management
Manages code history and versions.

GitHub

Remote code hosting
Hosts your code in the cloud.

Team-focused
Facilitates team collaboration and code sharing.

Online dependent
Requires an internet connection to use.

Code sharing
Enables sharing and reviewing code.

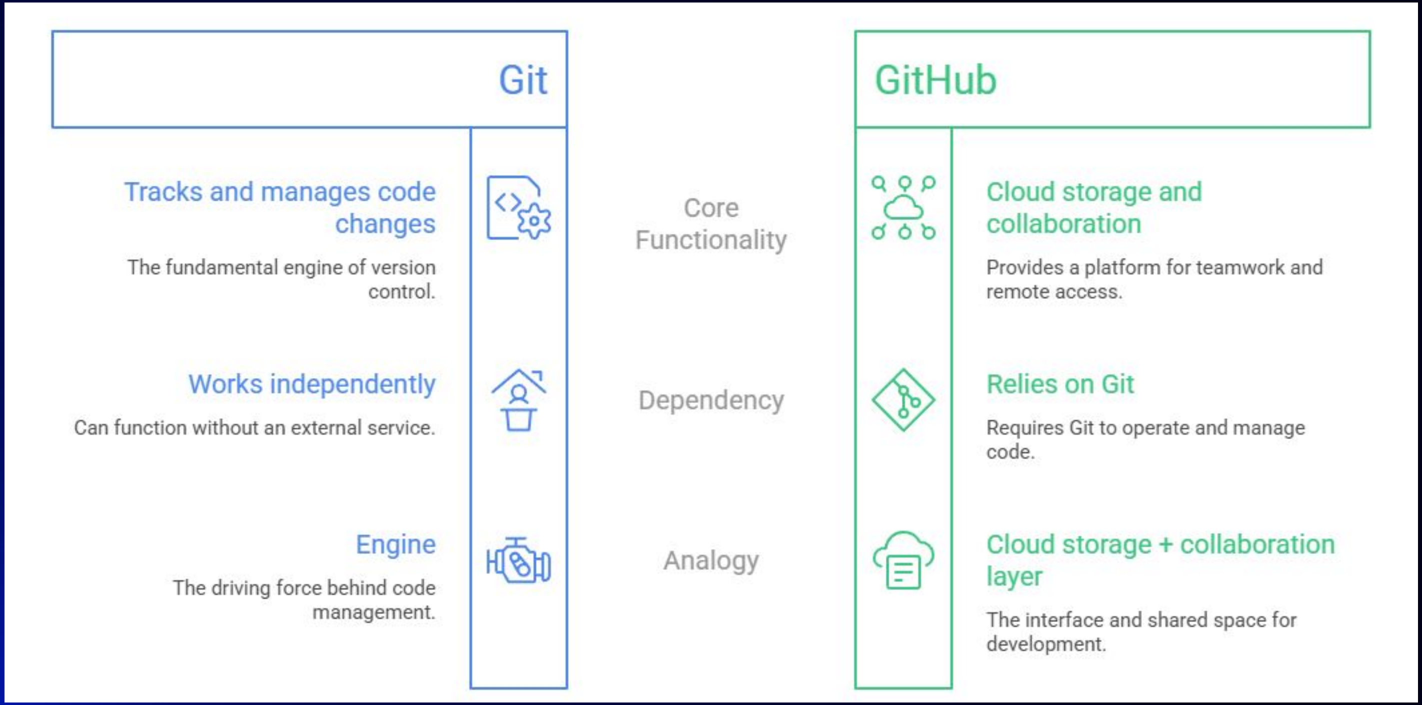
What is Git?

- Git is a version control system
- Helps track changes in code over time
- Works locally on your computer
- Allows saving versions (commits)
- Supports branching and merging
- Helps undo mistakes
- Works without internet and without GitHub

What is GitHub?

- GitHub is a cloud-based platform
- Used to store Git repositories online
- Enables collaboration with teams
- Allows code sharing and project backup
- Requires internet to access

Git vs GitHub (Simplified)



Pop Quiz

Q. In the analogy, Git is like ______

A

Engine

B

Cloud storage

Pop Quiz

Q. In the analogy, Git is like ______

A

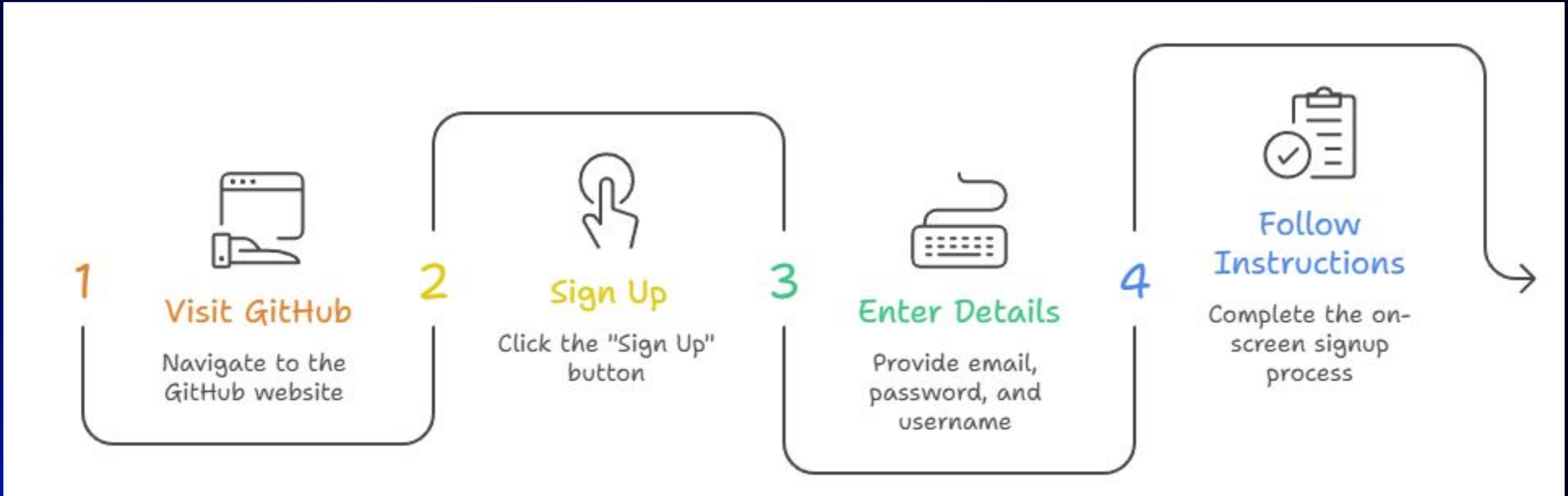
Engine

B

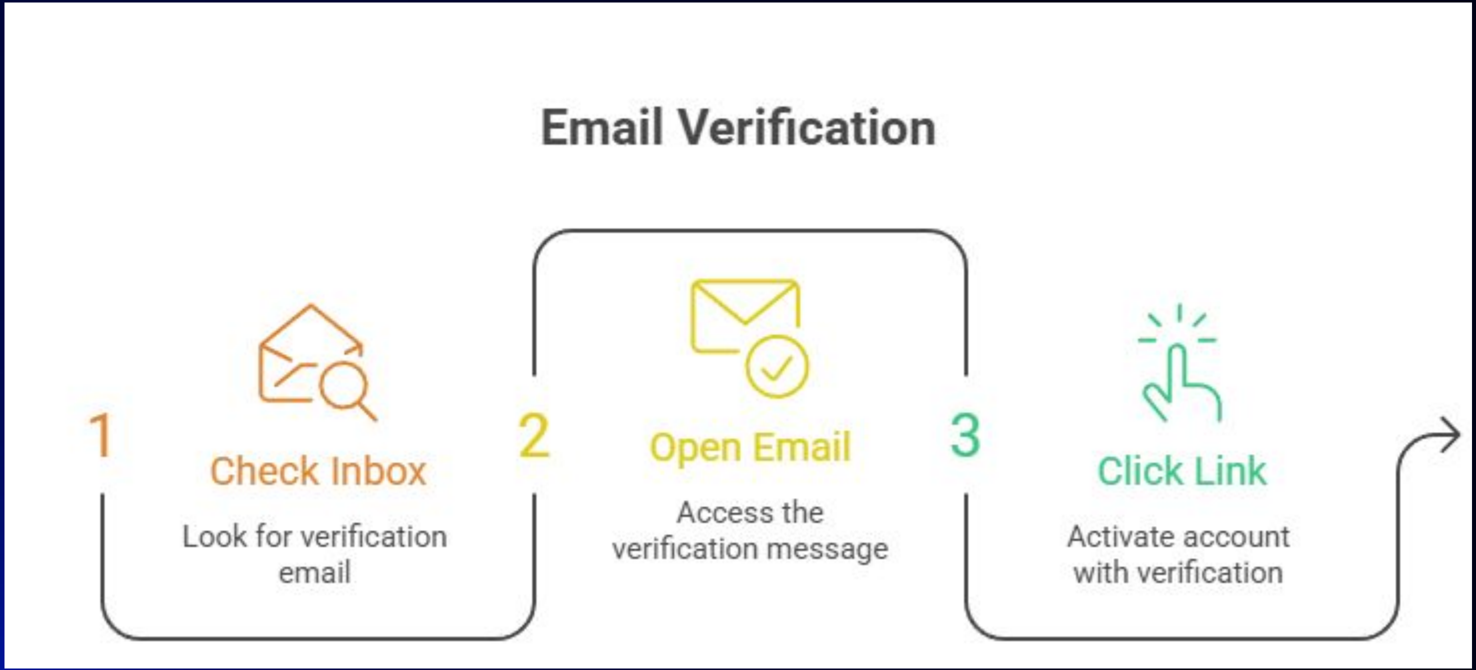
Cloud storage

GitHub Account & Profile Setup

Setting Up GitHub



Setting Up GitHub



Setting Up GitHub

4 **Log In Successfully**

Ensure you are logged into your GitHub account.

3 **Customize Profile**

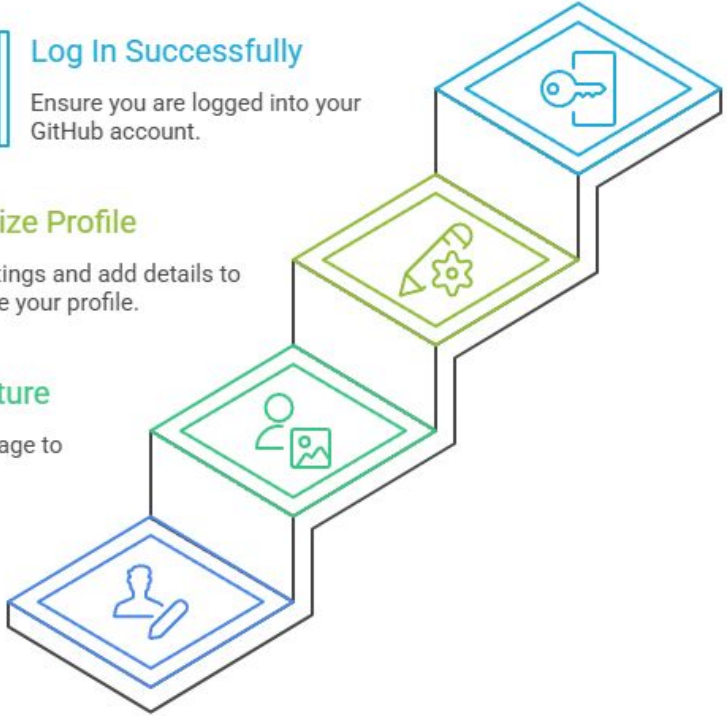
Adjust settings and add details to personalize your profile.

2 **Upload Profile Picture**

Optionally, upload an image to represent yourself.

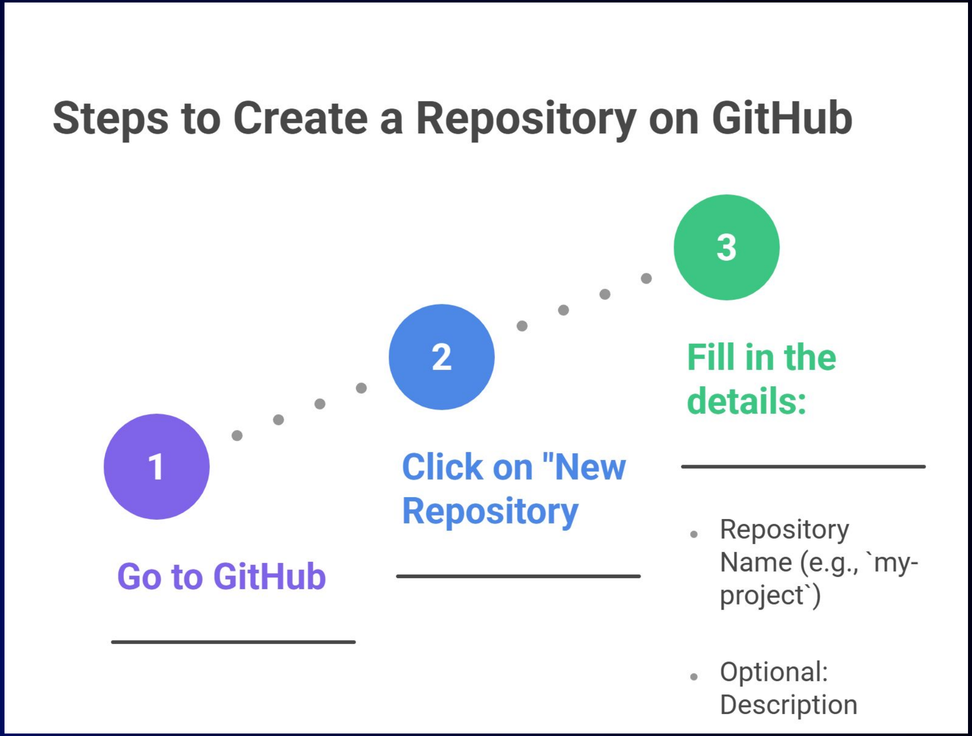
1 **Add Profile Name**

Enter your desired display name for your profile.



Creating a Remote Repository

Steps to Create a Repository on GitHub



Public vs Private Repository

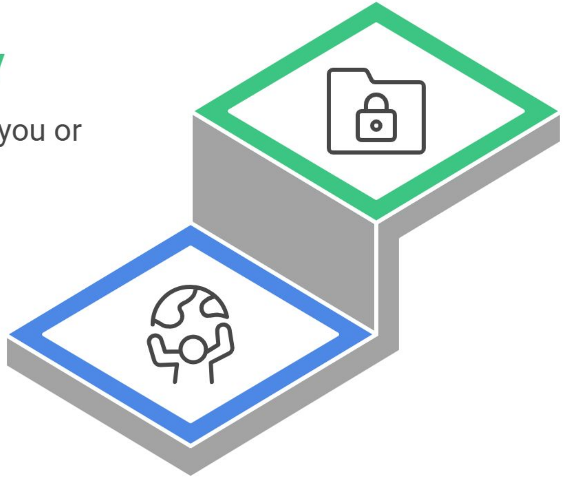
Choosing Repository Visibility

2 | **Private Repository**

Restricts access to only you or selected individuals.

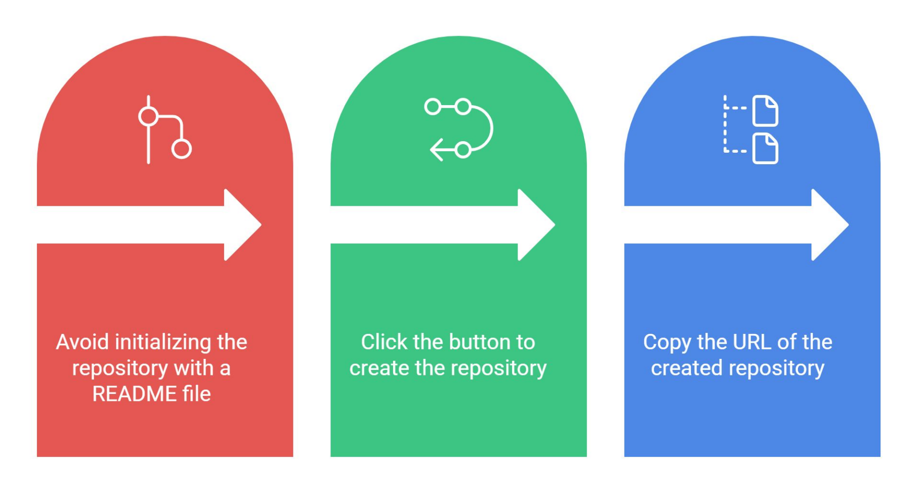
1 | **Public Repository**

Allows anyone to view and contribute to your code.



Important Setup Instructions

- ❌ Do NOT initialize with README
  a. Keeps repository empty for linking with local project
- ✅ Click "Create Repository"
- 📋 Copy the repository URL
  a. Example: https://github.com/username/repo.git
  b. Used to connect your local Git project



Pop Quiz

Q. What do you copy after creating a repository?

A

Repository name

B

Repository URL

Pop Quiz

Q. What do you copy after creating a repository?

A

Repository name

B

Repository URL

Connect Local Repo to GitHub

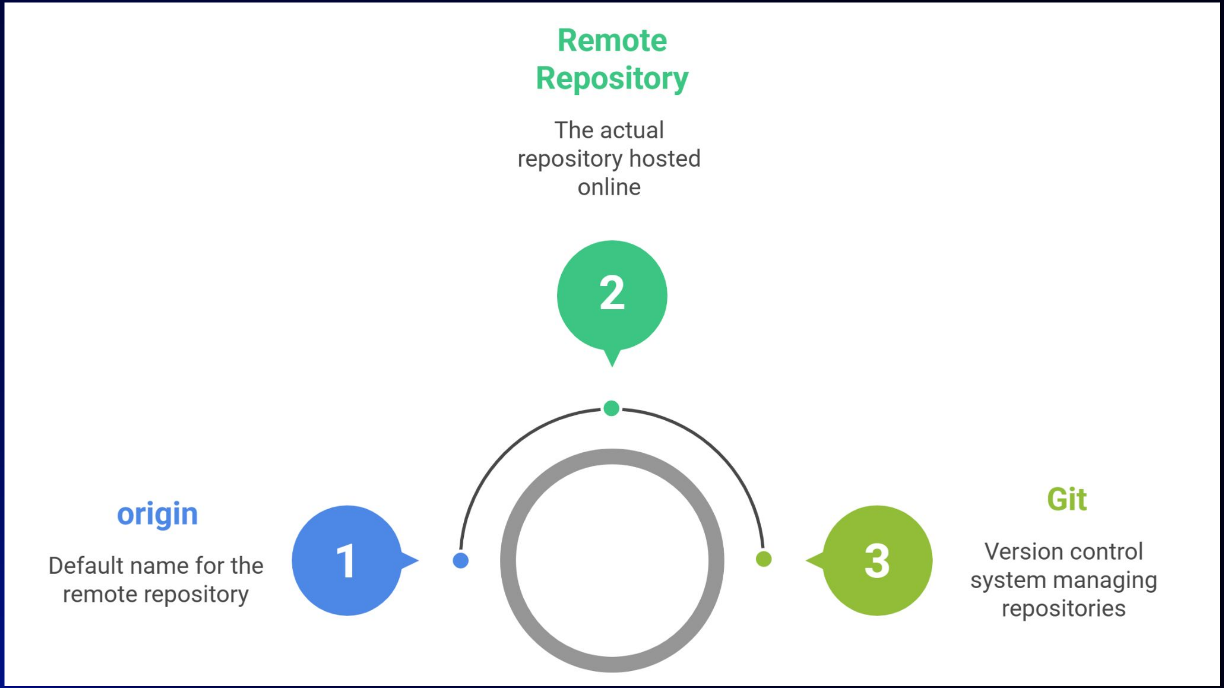
Add Remote Repository

- Open terminal inside your project folder
- Run the command:

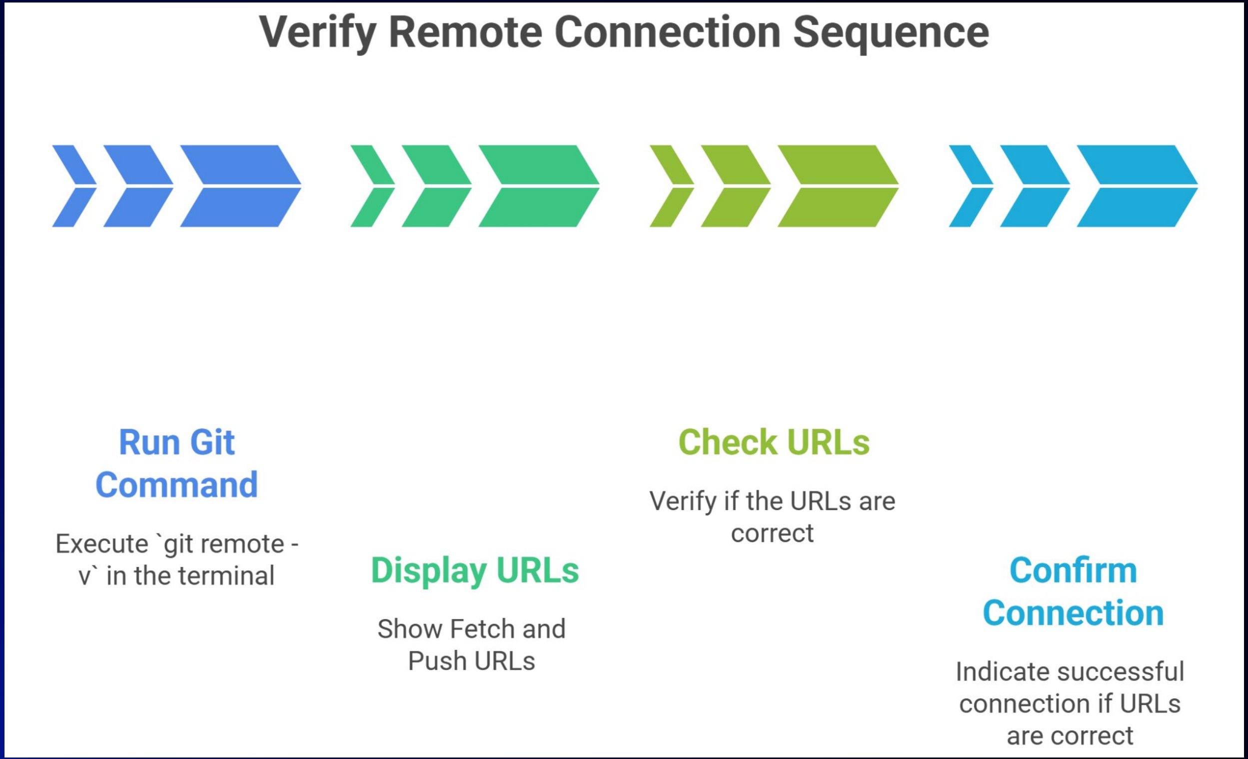
```
git remote add origin <repo-url>
```

- Replace <repo-url> with your GitHub repository link

What is “origin”?



Verify Remote Connection



Pop Quiz

Q. What does git remote -v show?

A

Commit history

B

Remote URLs for fetch and push

Pop Quiz

Q. What does git remote -v show?

A

Commit history

B

Remote URLs for fetch and push

Push Code to GitHub

Push Command

- Run the command:

  `git push -u origin main`

- **origin** → remote repository
- **main** → branch name

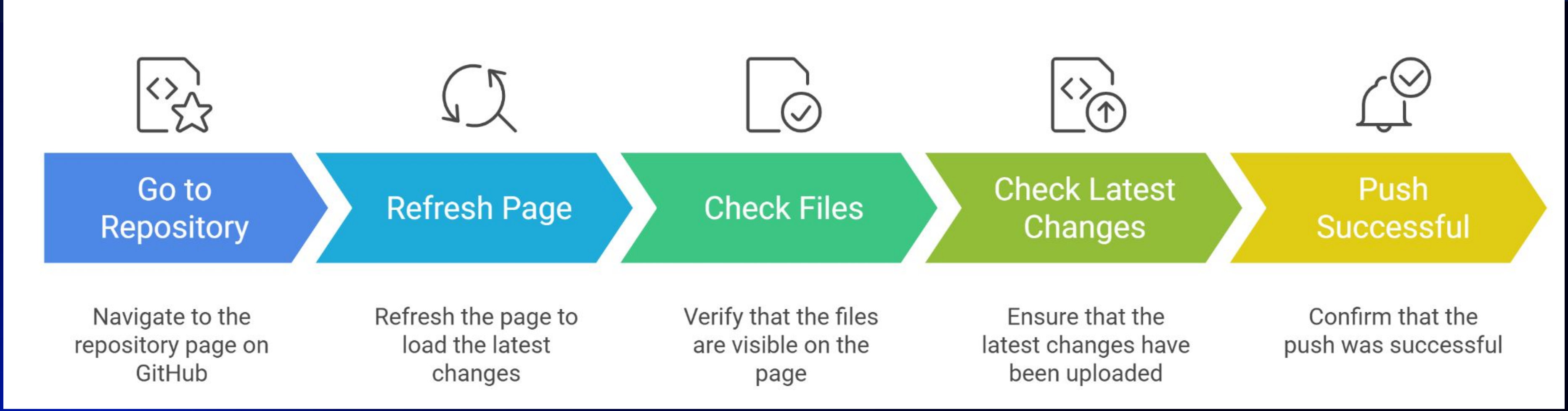
What Does “-u” Mean?

- -u stands for **upstream**
- It links your local branch (main) with remote branch
- After first push, you can simply use:

```
git push
```

- No need to write full command again

Verify on GitHub



Pop Quiz

Q. What does “origin” refer to in the push command?

A

Default remote repository name

B

Branch name

Pop Quiz

Q. What does “origin” refer to in the push command?

A

Default remote repository name

B

Branch name

Clone a Repository

Clone a Repository

- Cloning means copying a remote repository to your local machine
- It includes:
  a. All project files 📁
  b. Complete version history 🕒
  c. Commonly used to start working on an existing project from GitHub

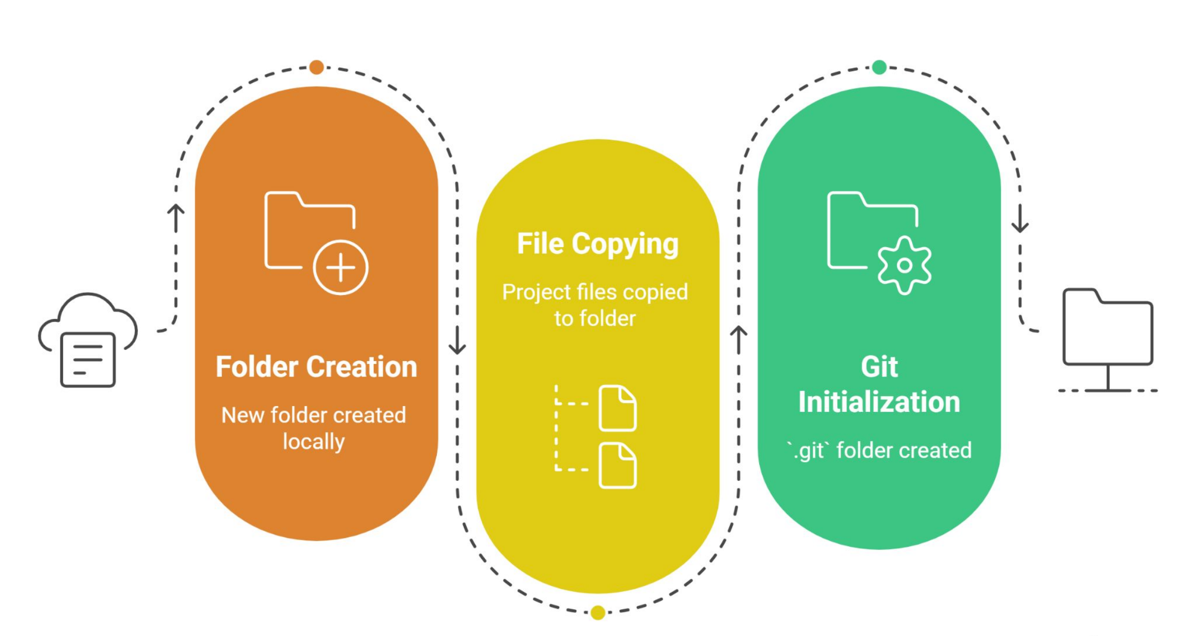
Clone Command

- Run the command:

  git clone <repo-url>

- Replace <repo-url> with the GitHub repository link
- Git will download the project to your system

What Happens After Cloning?



`.git` folder is created, tracks history of the project

Open the Cloned Project

- Navigate into the folder:

```
cd repository-name
```

- Open in your code editor (e.g., VS Code)
- Start working on the project 💻

Pop Quiz

Q. What does “cloning a repository” mean?

A

Cloning downloads the project along with its version history from GitHub.

B

Uploading files

Pop Quiz

Q. What does “cloning a repository” mean?

A

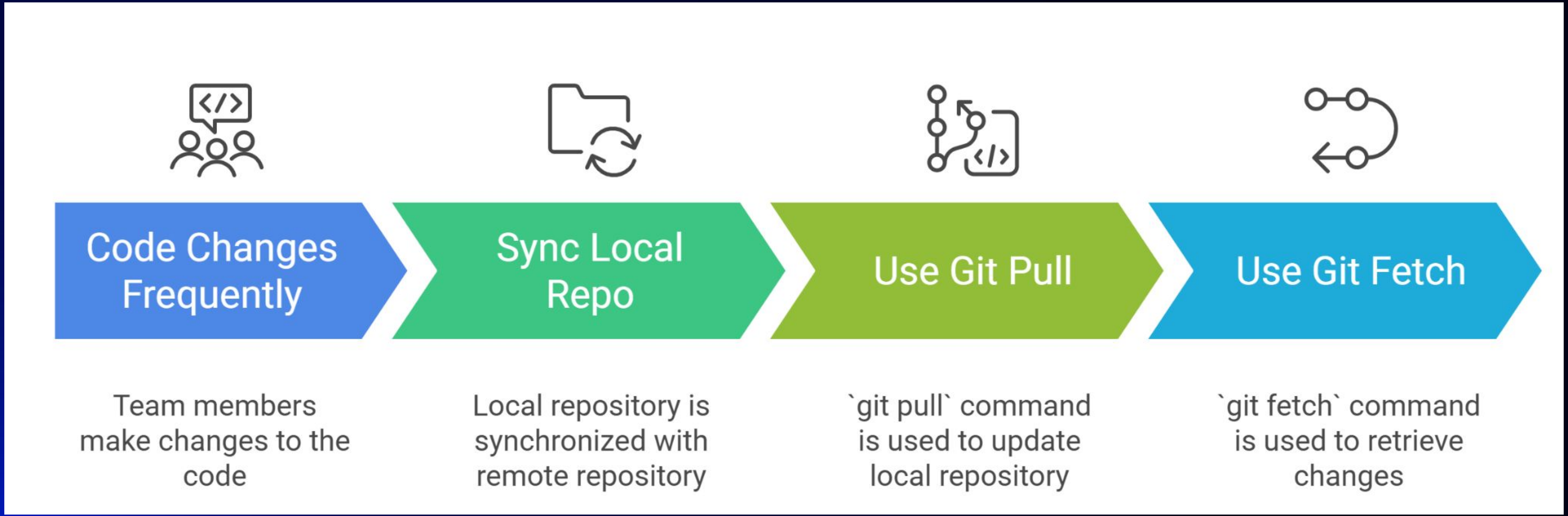
Cloning downloads the project along with its version history from GitHub.

B

Uploading files

Pull and Fetch (Keeping Code Updated)

Keeping Code Updated (Pull & Fetch)



git pull (Fetch + Merge)

- Command:

```
git pull
```

- What it does:
  - Downloads latest changes 📥
  - Automatically merges into your local branch
- Best for quick updates

git fetch (Only Download)

- Command:

```
git fetch
```

- What it does:
  - Downloads changes from remote
  - Does NOT merge automatically
- Useful when you want to review changes before merging

Simple Example

Teammate pushes new code to GitHub
You run:

- git fetch → see what changed 👀
- git pull → update your project

🔄 Keeps your code up-to-date and avoids conflicts

Pop Quiz

Q. Which command automatically updates your local branch?

A

git fetch

B

git pull

Pop Quiz

Q. Which command automatically updates your local branch?

A

git fetch

B

git pull

PW SKILLS

Take A 5-Minute Break!

- Stretch and relax
- Hydrate
- Clear your mind
- Be back in 5 minutes

Collaboration Workflow Using Branches

Collaboration Workflow Using Branches

In team projects, multiple developers work together 👥
Best practice:

- ❌ Do NOT work directly on `main`
- ✅ Use separate branches for features

Helps:

- Avoid conflicts
- Keep main branch stable

Commonly used on GitHub

Create a Feature Branch

- Create a new branch for your work:

```
git checkout -b feature-update
```

- feature-update = branch name (can be anything meaningful)
- Work independently without affecting main

Make Changes & Push Branch

- Steps:
  - Make changes ✏️
  - Commit changes:

```
git add .
git commit -m "Added new feature"
```

- Push branch to GitHub:

```
git push origin feature-update
```

View Branch on GitHub

Go to your repository on GitHub
You will see the new branch listed
From here:

- Create a Pull Request (PR)
- Merge changes into `main` after review

Pop Quiz

Q. What is the correct order after creating a branch?

A

Change → Commit → Push

B

Commit → Change → Push

Pop Quiz

Q. What is the correct order after creating a branch?

A

Change → Commit → Push

B

Commit → Change → Push

Class Project

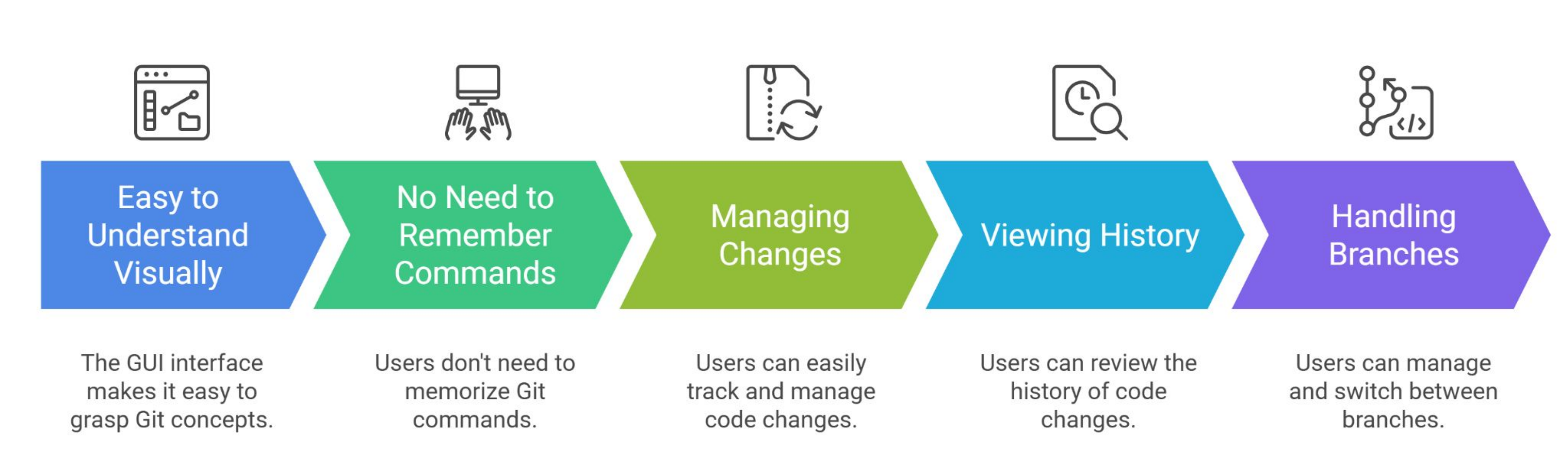
Basic Collaboration Flow (Without PR Deep Dive)

GitHub Desktop (Optional)

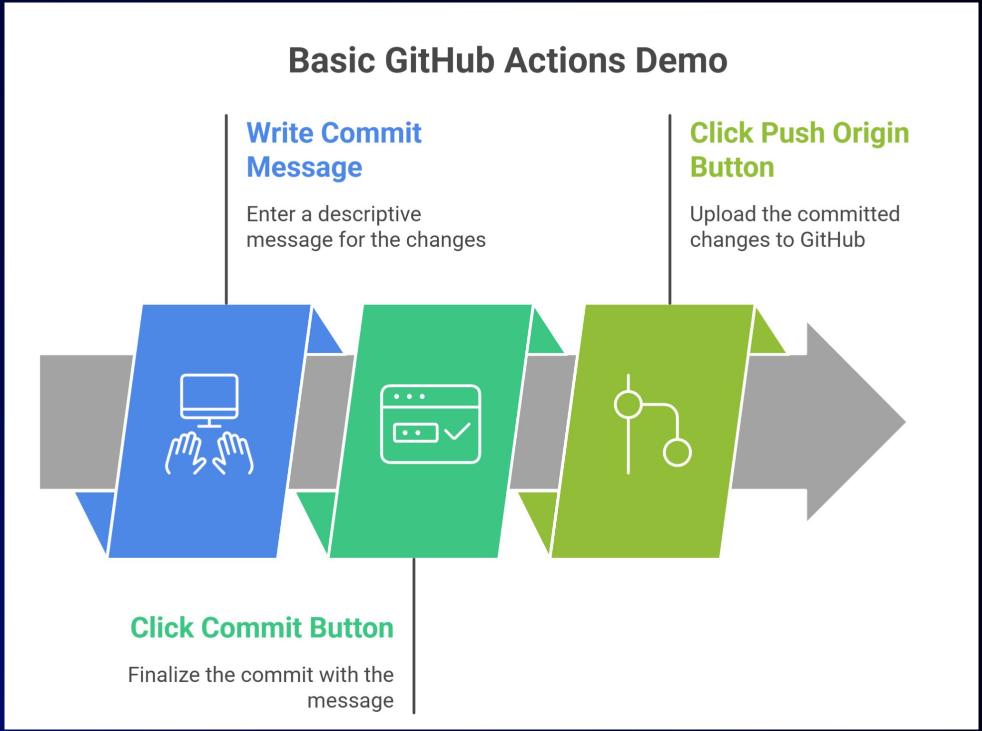
GitHub Desktop (Optional)

- GitHub Desktop is a GUI tool for using Git without commands
- Useful for beginners who prefer visual interface 🖥️
- Alternative to using terminal (CLI)

Why Use GitHub Desktop?



Basic Actions (Demo)



Important Note

CLI	GUI
Preferred for learning Git Offers a deeper understanding of Git's inner workings	**Helpful for Git operations** Provides a more intuitive and visual approach
Gives better understanding of how Git works Forces users to learn underlying commands and concepts	**Less direct understanding of Git's mechanics** Abstracts away the command-line details
Primary tool for learning and advanced use The foundation for mastering Git	**Support tool, not a replacement** Complements CLI, making certain tasks easier

Pop Quiz

Q. What is a limitation of using GUI tools like GitHub Desktop?

A

Deep understanding of Git concept

B

Less understanding of Git concepts

Summary

In this lecture, we have covered:

- Understood the difference between Git and GitHub
- Set up a GitHub account and basic profile
- Created and connected local repositories to remote repositories
- Learned how to push code and clone repositories
- Kept code updated using git pull and git fetch
- Understood basic collaboration using branches
- Explored GitHub Desktop (optional GUI tool)

DOUBT RESOLUTION

Important

- Complete the post-class assessment
- Complete assignments (if any)
- Practice the concepts and techniques taught in this session
- Review your lecture notes
- Note down questions and queries regarding this session and consult the teaching assistants.

THANK YOU